10 Alternatives to Vanilla Continual Finetuning

Vanilla continual fine-tuning often underperforms not because the model lacks capacity, but because sequential updates force an unstable compromise between *plasticity* and *retention*. As the model adapts to new data, it may either drift too far from previously useful representations or remain too constrained to meaningfully absorb the new distribution. This tension is especially acute in modern large models, where naive full-parameter updates are prone to *forgetting*, *overspecialization*, and inefficient use of adaptation capacity. **Accordingly, recent work has moved beyond plain continual fine-tuning toward more structured variants** that preserve useful prior knowledge while still enabling targeted adaptation. In this section, we focus on ***three particularly promising directions***: ***slow–fast parameter-efficient tuning***, ***replay with selective distillation***, and ***shared adapter-based continual tuning***. Together, they represent some of the strongest recent alternatives to vanilla continual fine-tuning when one wishes to improve continual adaptation ***without redesigning the base model itself***.

10.1 Slow-Fast Parameter-Efficient Tuning (SAFE)

Motivation. Vanilla continual fine-tuning tends to fail for two coupled reasons: *(i)* direct adaptation to the first session can overwrite useful general knowledge inherited from the pre-trained model, and *(ii)* freezing the adapted model in later sessions suppresses the plasticity required to absorb genuinely novel concepts. SAFE (**Slow And Fast parameter-Efficient tuning**) addresses this by maintaining two complementary adaptation pathways: a ***slow learner*** that is calibrated once to preserve and inherit the broad generalization of the pre-trained model, and a ***fast learner*** that continues to adapt in subsequent sessions while being explicitly regularized by the slow branch (Zhao et al., 2024).

Set-up. We consider a replay-free class-incremental stream of sessions

$$\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_T,$$

with disjoint label sets

$$\mathcal{Y}_i \cap \mathcal{Y}_j = \varnothing \qquad \text{for } i \neq j.$$

Let the frozen pre-trained model (PTM) feature extractor be denoted by

$$\phi_{\text{PTM}}(x) \in \mathbb{R}^d.$$

SAFE introduces two parameter-efficient tuned feature extractors:

$$\phi_{\text{slow}}(x), \qquad \phi_{\text{fast}}(x) \in \mathbb{R}^d,$$

implemented via a lightweight PET mechanism such as ***Adapters***, ***SSF***, or ***VPT*** (Houlsby et al., 2019; Lian et al., 2022; Jia et al., 2022; Zhao et al., 2024). Their corresponding classifiers at session t are

$$W_{\text{slow}},\ W_{\text{fast}} \in \mathbb{R}^{d \times |\mathcal{Y}_{1:t}|}.$$

Core idea. The design principle is simple:

> ***learn slowly where generality matters, and learn fast where novelty matters.***

The slow learner is tuned only in the first session and then frozen; the fast learner is updated in each subsequent session. During inference, both are combined adaptively.

10.2 Slow-Fast Parameter-Efficient Tuning (SAFE)

Why SAFE is attractive here. Our immediate goal is ***not*** to redesign the underlying JEPA objective, nor to introduce explicit factorized heads at this stage. Instead, we seek a stronger ***continual fine-tuning protocol*** that can be layered on top of the existing V-JEPA training recipe. In this regard, ***SAFE*** (**S**low and **F**ast parameter-**E**fficient tuning) provides a compelling template: it explicitly separates a ***stable adaptation pathway*** from a ***plastic adaptation pathway***, thereby addressing the central tension of continual fine-tuning—namely, that the model must absorb new-domain structure without catastrophically drifting away from previously useful representations (Zhao et al., 2024).

Our use of SAFE. We do ***not*** adopt SAFE here as a wholesale replacement of the JEPA objective. Rather, we use it in the following spirit:

1. preserve V-JEPA's original predictive loss as the ***primary learning signal***,
2. use SAFE's ***slow/fast decomposition*** to determine *which parameters are updated, when, and how fast*,
3. optionally incorporate lightweight stabilizers only if needed, but avoid turning the training objective into a fundamentally different task.

Thus, SAFE is best viewed here as a ***structured continual adaptation wrapper around V-JEPA***, not as a replacement for JEPA itself.

10.2.1 Base V-JEPA objective remains unchanged

Let x denote an input video clip, M_c the context mask, M_t the target mask, f_θ the online encoder, $f_{\bar{\theta}}$ the target / momentum encoder, and g_ϕ the predictor. Then the original V-JEPA-style predictive objective can be written abstractly as

$$\mathcal{L}_{\text{JEPA}} = \mathbb{E}_x \Big[\| g_\phi(f_\theta(M_c \odot x), M_t) - \text{sg}(f_{\bar{\theta}}(M_t \odot x)) \|_2^2 \Big],$$

where $\text{sg}(\cdot)$ denotes stop-gradient. The key design decision in this section is that ***the above JEPA objective remains the principal optimization target throughout continual adaptation.***

This point is important: our aim is to improve ***continual fine-tuning dynamics***, not to alter the basic predictive semantics of V-JEPA.

10.2.2 SAFE as a slow-fast update protocol

SAFE was originally introduced for continual learning with pre-trained models under a parameter-efficient tuning (PET) setting. Its core insight is that a single adaptation pathway is often asked to do two incompatible jobs: preserve broad generalization inherited from pre-training, and rapidly incorporate genuinely novel concepts from later sessions (Zhao et al., 2024). SAFE resolves this by introducing:

- a ***slow learner*** that is calibrated early and then kept stable, and
- a ***fast learner*** that remains updateable and is responsible for later-session plasticity.

We adapt this idea to V-JEPA as follows.

Backbone decomposition. Let the pre-trained V-JEPA parameters be partitioned into a frozen shared backbone and lightweight trainable PET modules:

$$\Theta = \Theta_{\text{shared}} \cup \Theta_{\text{S-PET}} \cup \Theta_{\text{F-PET}}.$$

Here:

- Θ_{shared} denotes the original V-JEPA backbone parameters, largely frozen or updated extremely conservatively;
- $\Theta_{\text{S-PET}}$ denotes the ***slow PET*** parameters;
- $\Theta_{\text{F-PET}}$ denotes the ***fast PET*** parameters.

In practice, these PET blocks may be instantiated via ***Adapters***, ***LoRA-like modules***, ***SSF***, or related lightweight tuning blocks. SAFE itself is presented as a PET-based framework and illustrates Adapter, SSF, and VPT-style blocks as canonical choices (Zhao et al., 2024).

10.2.3 Stage I: slow adaptation on the first continual session

Suppose the first continual session is $\mathcal{D}_1$. We initialize from a pre-trained V-JEPA checkpoint and attach a slow PET branch:

$$f_\theta^{(S)} = f_{\Theta_{\text{shared}}, \Theta_{\text{S-PET}}}.$$

During this stage, we optimize primarily with the original JEPA objective:

$$\min_{\Theta_{\text{S-PET}}, \phi} \quad \mathcal{L}_{\text{JEPA}}(\mathcal{D}_1).$$

Why only the slow branch first? The first session is special: it determines the initial domain adaptation away from the foundation checkpoint. If we fully fine-tune the entire model here, the resulting representation may overfit to the first continual domain and lose the broad generalization inherited from pre-training. SAFE's central argument is precisely that ***direct first-session tuning can overwrite general PTM knowledge***, especially when the first session is much smaller or narrower than the original pre-training distribution (Zhao et al., 2024). The slow branch addresses this by making early adaptation ***controlled, lightweight, and structurally constrained***.

Optional SAFE-style transfer regularization. In the original SAFE formulation, the slow learner in the first session is additionally regularized by a transfer loss based on the correlation between the pre-trained model features and the PET-adapted model features (Zhao et al., 2024). If desired, this can be added here in a lightweight way:

$$\mathcal{L}_{\text{slow}} = \mathcal{L}_{\text{JEPA}} + \lambda_{\text{tr}} \, \mathcal{L}_{\text{transfer}},$$

where $\mathcal{L}_{\text{transfer}}$ encourages the adapted representation to remain aligned with the original checkpoint. However, for our present purpose, this term is ***optional rather than central***; the main point is that the ***JEPA loss remains dominant***.

After Stage I, we freeze $\Theta_{\text{S-PET}}$, yielding a stable slow branch.

10.2.4 Stage II: fast adaptation for later continual sessions

For each subsequent session $t > 1$, we retain the frozen slow branch and add or continue a fast PET pathway:

$$f_\theta^{(F)} = f_{\Theta_{\text{shared}}, \Theta_{\text{F-PET}}}.$$

The slow branch acts as a ***stable memory anchor***, while the fast branch remains plastic and absorbs new session-specific structure.

The cleanest version for our setting is to optimize the fast branch still using the same JEPA loss:

$$\min_{\Theta_{\text{F-PET}}, \phi} \quad \mathcal{L}_{\text{JEPA}}(\mathcal{D}_t), \qquad t > 1.$$

Interpretation. This stage changes *which parameters move*, not *what the model is asked to predict*. The predictive task remains JEPA-style masked / future embedding prediction; the novelty lies in assigning that adaptation burden to a dedicated ***fast subspace*** rather than rewriting the whole model.

10.2.5 A minimal SAFE-for-V-JEPA formulation

Putting the above together, our preferred minimalist SAFE instantiation is:

Session $t = 1$:

$$\mathcal{L}^{(1)} = \mathcal{L}_{\text{JEPA}}(\mathcal{D}_1) \quad \text{optimized over } \Theta_{\text{S-PET}} \text{ and predictor parameters.}$$

Sessions $t > 1$:

$$\mathcal{L}^{(t)} = \mathcal{L}_{\text{JEPA}}(\mathcal{D}_t) \quad \text{optimized over } \Theta_{\text{F-PET}} \text{ and predictor parameters, with } \Theta_{\text{S-PET}} \text{ frozen.}$$

This yields a very important property:

$$\underbrace{\text{objective}}_{\text{unchanged}} \quad \text{vs.} \quad \underbrace{\text{adaptation pathway}}_{\text{changed}}.$$

That is the main reason SAFE is appealing here.

10.2.6 Selective unfreezing and slow–fast learning rates

SAFE also suggests a useful ***optimization schedule*** even when one does not import all of its auxiliary losses. Concretely, we can assign distinct update rates to different parameter groups:

$$\eta_{\text{shared}} \ll \eta_{\text{S-PET}} \ll \eta_{\text{F-PET}},$$

or, in an even more conservative setting,

$$\eta_{\text{shared}} \approx 0.$$

Meaning of the above hierarchy. Lower-level shared representation blocks are either frozen or updated only minimally; the slow PET branch moves modestly and only in the earliest stage; the fast PET branch remains the main adaptation carrier in later sessions. This induces a ***slow–fast stability–plasticity hierarchy*** without forcing us to redefine the self-supervised objective.

A related practical variant is ***selective unfreezing***:

- first update only predictor + PET blocks,
- then, only if needed, unfreeze the top K transformer blocks at a much smaller learning rate,
- keep the lower visual representation largely fixed.

Again, this is a training *protocol* change, not a task change.

10.2.7 Optional lightweight alignment to the frozen slow branch

If later experiments show that the fast branch drifts too aggressively, we can add a very light alignment term to the frozen slow branch:

$$\mathcal{L}_{\text{align}} = \mathbb{E}_{x\sim\mathcal{D}_t}\left[1 - \cos\Big(h^{(F)}(x),\, h^{(S)}(x)\Big)\right],$$

where $h^{(F)}(x)$ and $h^{(S)}(x)$ denote the latent context embeddings produced by the fast and slow branches, respectively. Then the total loss becomes

$$\mathcal{L}^{(t)}_{\text{total}} = \mathcal{L}_{\text{JEPA}}(\mathcal{D}_t) + \lambda_{\text{align}}\,\mathcal{L}_{\text{align}}.$$

Crucially, this remains a mild regularizer. The conceptual center of gravity stays with $\mathcal{L}_{\text{JEPA}}$, unlike the full SAFE formulation where cross-classification and transfer losses play a larger methodological role.

10.2.8 Why SAFE is a good fit for this paper

SAFE is particularly appropriate for our current stage for four reasons.

1. ***It is implementation-friendly.*** SAFE has a public implementation, which lowers the barrier to reproducing and adapting the slow/fast training scaffold in our codebase.
2. ***It addresses continual fine-tuning directly.*** The method was designed precisely for scenarios where naive PET or continual adaptation either overwrites useful pre-trained knowledge in the first session or becomes too rigid in later sessions.
3. ***It is compatible with keeping V-JEPA intact.*** The central slow/fast idea does not require us to abandon JEPA-style predictive learning; it mainly changes the parameterization and scheduling of continual adaptation.
4. ***It gives a principled path beyond vanilla continual fine-tuning.*** Rather than asking one monolithic parameter set to simultaneously preserve past knowledge and absorb future novelty, SAFE introduces a structured separation of roles.

10.2.9 How we will use it in practice

Our concrete plan is therefore the following:

1. start from a strong pre-trained V-JEPA checkpoint;
2. attach PET modules to the predictor and upper transformer blocks;
3. train a ***slow branch*** on the first continual session using the original JEPA loss;
4. freeze that slow branch thereafter;
5. continue later sessions through a ***fast branch*** trained again with the same JEPA objective;
6. use differential learning rates / selective unfreezing to ensure that high-level predictive components adapt faster than lower-level representation blocks;
7. add lightweight alignment only if empirical drift remains a problem.

In short. For us, SAFE is best understood not as "replace V-JEPA with a new loss," but as

> ***keep the JEPA objective, but make continual fine-tuning structurally asymmetric.***

The slow branch preserves early adapted knowledge; the fast branch retains plasticity for future sessions; and the underlying self-supervised predictive task remains unchanged.

Bottom line. This makes SAFE an especially attractive first upgrade over vanilla continual fine-tuning: it is ***practical***, ***available***, ***parameter-efficient***, and conceptually aligned with our immediate goal of improving continual adaptation ***without yet altering the core V-JEPA learning objective*** (Zhao et al., 2024).

10.3 Replay + Selective Distillation (SEEKR)

Why SEEKR is attractive here. If SAFE addresses the stability–plasticity tension by separating ***how*** adaptation is parameterized, then ***SEEKR*** addresses it by changing ***what must be preserved*** during continual updating. Its starting point is both simple and important: ***replay is already a strong continual learning baseline, but naïve replay is often inefficient because it treats all retained information as equally valuable.*** SEEKR improves this by pairing replay with ***selective distillation***: rather than distilling the entire previous model uniformly, it identifies the most valuable internal components for knowledge retention and distills those preferentially. In the original EMNLP 2024 formulation, these components are ***attention heads*** in large language models, selected using ***forgettability-based*** and ***task-sensitivity-based*** importance measures. The paper reports improved performance and efficiency over prior replay-based methods, including strong results with much smaller replay budgets.

Our use of SEEKR. As with SAFE, we do ***not*** import SEEKR into our setting as a literal architectural transplant. SEEKR was proposed for continual learning of LLMs and performs ***attention-head distillation***; our setting is V-JEPA, not autoregressive language modeling. Accordingly, we adopt the ***principle*** of SEEKR, not its exact modality-specific mechanism:

1. preserve V-JEPA's original predictive objective as the ***main learning signal***,
2. introduce ***replay*** to anchor previously learned representations,
3. add ***selective distillation*** only on the most retention-critical internal quantities, rather than distilling the whole model indiscriminately.

Thus, SEEKR becomes in our setting a ***replay-centric continual adaptation strategy with targeted retention***, rather than a full redesign of the underlying JEPA objective.

10.3.1 Base V-JEPA objective remains primary

Let x denote an input video clip, M_c the context mask, M_t the target mask, f_θ the online encoder, $f_{\bar{\theta}}$ the target / momentum encoder, and g_ϕ the predictor. As before, the base V-JEPA predictive objective is

$$\mathcal{L}_{\text{JEPA}} = \mathbb{E}_x \Big[\| g_\phi(f_\theta(M_c \odot x), M_t) - \text{sg}(f_{\bar{\theta}}(M_t \odot x)) \|_2^2 \Big].$$

Our core decision is that this loss remains ***the principal optimization target*** during continual training. Replay and distillation are introduced only to stabilize continual adaptation, not to replace the main self-supervised prediction task.

10.3.2 Replay as the first stabilization mechanism

Suppose the continual stream is partitioned into sessions

$$\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_T.$$

At session t, instead of fine-tuning only on the current data $\mathcal{D}_t$, we maintain a replay memory

$$\mathcal{M}_{t-1} = \{x^{(1)}, x^{(2)}, \ldots, x^{(m)}\}$$

containing a compact set of retained examples from previous sessions. The most basic replay-augmented objective is then

$$\mathcal{L}_{\text{replay}} = \mathcal{L}_{\text{JEPA}}(\mathcal{B}_{\text{new}}) + \lambda_{\text{rep}} \mathcal{L}_{\text{JEPA}}(\mathcal{B}_{\text{mem}}),$$

where $\mathcal{B}_{\text{new}} \sim \mathcal{D}_t$ and $\mathcal{B}_{\text{mem}} \sim \mathcal{M}_{t-1}$.

Why replay matters. Pure sequential fine-tuning updates the model using only gradients from the current session, which allows the representation to drift toward the newest domain. Replay counteracts this by keeping older regions of the learned representation manifold ***active*** during optimization. SEEKR begins from exactly this observation: replay is already an effective continual learning tool, but its full potential is often not realized because retention is treated too coarsely.

10.3.3 Why selective distillation is better than uniform distillation

A natural next step after replay is distillation from the previous checkpoint. Let $(\theta^{\text{old}}, \phi^{\text{old}})$ denote the frozen model from the end of session $t-1$. Uniform feature distillation would take a form such as

$$\mathcal{L}_{\text{distill}}^{\text{uniform}} = \mathbb{E}_{x \sim \mathcal{M}_{t-1}} [\| h_\theta(x) - h_{\theta^{\text{old}}}(x) \|_2^2],$$

where $h_\theta(x)$ denotes some internal representation.

The problem is that this objective implicitly assumes ***all internal components are equally important for retention***. SEEKR argues that this assumption is wasteful: some components contribute much more strongly than others to preserving prior knowledge, and distilling everything uniformly both increases cost and dilutes the retention signal. Instead, SEEKR identifies ***the most valuable components*** and distills only those. In the original paper, these are attention heads chosen using ***forgettability*** and ***task sensitivity*** criteria, followed by a hierarchical budget allocation strategy so that replay and distillation remain data-efficient.

The philosophical shift. This is the key SEEKR idea that transfers to our setting:

> ***Retention should be selective, not indiscriminate.***

The goal is not to freeze the whole past model in amber, but to preserve the ***parts of the computation most responsible for previously acquired knowledge***.

10.3.4 Translating SEEKR to V-JEPA

Because V-JEPA is not a language model, attention-head distillation is not the only or even necessarily the best unit of retention. What transfers more faithfully is the ***selection principle***. Concretely, in our setting the selectively distilled units may be any of the following:

- selected encoder blocks,

- selected predictor layers,
- selected latent feature channels,
- selected spatial–temporal tokens,
- selected attention maps or cross-token interactions,
- selected predictor outputs on replay clips.

Let $\mathcal{S}_t$ denote the selected set of retention-critical components at session t. Then a generic selective distillation loss takes the form

$$\mathcal{L}_{\text{sel-distill}} = \sum_{u \in \mathcal{S}_t} \alpha_u \, d\Big(z_u^{\text{new}}(x), z_u^{\text{old}}(x)\Big), \qquad x \sim \mathcal{M}_{t-1},$$

where

- $z_u^{\text{new}}(x)$ is the current value of component u,
- $z_u^{\text{old}}(x)$ is the corresponding value from the frozen old model,
- $d(\cdot,\cdot)$ may be an L_2, cosine, or KL-type discrepancy,
- α_u weights each selected component by its estimated retention importance.

The total objective is then

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{JEPA}}(\mathcal{B}_{\text{new}}) + \lambda_{\text{rep}} \mathcal{L}_{\text{JEPA}}(\mathcal{B}_{\text{mem}}) + \lambda_{\text{sd}} \mathcal{L}_{\text{sel-distill}}.$$

What remains unchanged. Importantly, the first two terms are still pure JEPA-style predictive learning. The selective distillation term acts only as a ***retention regularizer*** on replayed content. This is exactly why SEEKR fits the philosophy of the current section better than heavier continual-learning methods that redefine the core training task.

10.3.5 How should the selected components be chosen?

SEEKR uses two complementary ideas to identify valuable attention heads:

1. ***Forgettability-based importance***: components whose behavior changes strongly across sessions are treated as especially important to preserve;
2. ***Task-sensitivity-based importance***: components that are especially responsive to task-relevant inputs are prioritized for distillation.

It then allocates replay and distillation budget hierarchically so that the most valuable units are protected first.

In our V-JEPA adaptation, we can retain exactly the same logic while redefining the unit of selection. For example, let u index predictor channels or latent tokens. A simple forgettability score could be

$$F_u = \mathbb{E}_{x \sim \mathcal{M}_{t-1}}[d\Big(z_u^{(t)}(x), z_u^{(t-1)}(x)\Big)],$$

while a task-sensitivity score could be

$$S_u = \mathbb{E}_{x \sim \mathcal{D}_t}[\|z_u(x)\|] \quad \text{or} \quad \mathbb{E}_{x \sim \mathcal{D}_t}[\|\nabla_{z_u} \mathcal{L}_{\text{JEPA}}(x)\|].$$

We then choose

$$\mathcal{S}_t = \text{TopK}_u \left(\beta_F F_u + \beta_S S_u\right),$$

and distill only those units.

Why this is philosophically right for us. This keeps the method aligned with SEEKR's central message while respecting modality differences:

> ***do not preserve everything; preserve what is most likely to be forgotten and most likely to matter.***

10.3.6 Why SEEKR is especially appealing when replay budget is small

One of the strongest practical messages of SEEKR is that ***better retention does not necessarily require more replay data***. The method is explicitly motivated by the observation that prior replay-based distillation methods often require relatively large replay buffers, whereas selective distillation can retain knowledge much more efficiently by spending the budget on the right internal units. The paper reports strong gains in both performance and efficiency, including settings with sharply reduced replay size.

This is particularly relevant for us because replay in video representation learning can be expensive:

- clips occupy more memory than text samples,
- temporal replay increases I/O and storage pressure,
- full-sequence distillation is costly at scale.

A SEEKR-style strategy is therefore attractive precisely because it says:

> ***use a small replay buffer, but make every replayed sample count more by distilling only the most retention-critical structure.***

10.3.7 How we will use it in practice

Our concrete SEEKR-inspired plan is the following:

1. maintain a compact replay buffer of clips from earlier continual sessions;
2. continue optimizing the standard V-JEPA loss on both current and replayed batches;
3. keep a frozen teacher checkpoint from the previous session;
4. define a small set of retention-critical internal units—initially predictor features or upper-layer latent tokens;
5. compute simple forgettability and sensitivity scores to select the top-K units;
6. distill only those units on replayed clips;
7. increase or decrease the selective distillation weight depending on observed drift.

This yields a natural progression:

$$\text{vanilla continual FT} \longrightarrow \text{replay} \longrightarrow \text{replay + selective distillation.}$$

The first step adds memory; the second step makes that memory ***targeted***.

10.3.8 Why SEEKR is a good fit for this paper

SEEKR is a strong fit for the present stage for four reasons.

1. ***It preserves the main JEPA objective.*** Replay and selective distillation regularize continual learning without changing the underlying prediction task.
2. ***It is more efficient than uniform retention.*** Rather than trying to preserve the entire previous model equally, it directs retention budget toward the most important internal components.
3. ***It is especially attractive under small replay budgets.*** This is highly relevant in video settings where replay is memory- and compute-intensive.
4. ***It gives a clean conceptual complement to SAFE.*** SAFE changes the *adaptation pathway*; SEEKR changes the *retention target*. Together, they represent two different but compatible routes beyond vanilla continual fine-tuning.

In short. For us, SEEKR is best understood not as "do attention-head distillation because the original paper did," but as

> ***keep the JEPA objective, add replay, and distill only the internal structure that matters most for retaining prior knowledge.***

Bottom line. This makes SEEKR an especially compelling second alternative to vanilla continual fine-tuning: it is ***replay-compatible***, ***data-efficient***, and philosophically well aligned with our goal of improving continual adaptation ***without abandoning the core V-JEPA learning objective***. The exact distillation unit will differ from the original LLM setting, but the central lesson carries over directly: ***effective retention is selective retention***.

10.4 Shared Adapter Continual Tuning (SSIAT / SAPT)

Why shared adapter continual tuning is attractive here. If SAFE separates continual adaptation into ***slow*** and ***fast*** pathways, and SEEKR improves ***what is retained*** through replay and selective distillation, then ***shared adapter continual tuning*** asks a different question: ***can we keep the backbone largely fixed and let a compact, reusable adaptation space absorb the continual shift?*** Recent work in both vision and language suggests that the answer is often yes. In vision, ***SSIAT*** (CVPR 2024) revisits parameter-efficient tuning in continual learning and reports that ***adapter tuning outperforms prompt-based methods***, even without expanding parameters at every session. In language, ***SAPT*** (ACL 2024) argues that continual PEFT systems should not treat *learning* task-specific PET blocks and *selecting* them at inference as two disconnected processes; instead, they should be aligned through a ***shared attentive learning-and-selection mechanism***. Together, these papers motivate a practical philosophy: ***do not continually rewrite the full model; continually refine a structured adapter space.***

Our use of SSIAT / SAPT. As with the previous two methods, we do ***not*** import these methods verbatim into V-JEPA. SSIAT is proposed in the class-incremental vision setting, and SAPT is developed for continual learning of large language models. What we borrow is the ***shared-adaptation principle***:

1. preserve V-JEPA's original predictive loss as the ***primary training signal***,
2. replace full continual fine-tuning with ***shared parameter-efficient modules*** that persist across sessions,
3. let continual adaptation happen mainly by updating this ***reusable adapter subspace*** rather than the whole backbone.

Thus, shared adapter continual tuning becomes in our setting a ***parameter-efficient continual fine-tuning strategy*** that changes *where* the model adapts, while leaving the underlying JEPA learning problem intact. SSIAT explicitly argues for incrementally tuning a *shared adapter* without per-session parameter expansion, and SAPT explicitly frames continual PEFT as a joint learning-and-selection problem over reusable PET blocks.

10.4.1 Base V-JEPA objective remains primary

Let x denote an input video clip, M_c the context mask, M_t the target mask, f_θ the online encoder, $f_{\bar{\theta}}$ the target / momentum encoder, and g_ϕ the predictor. As before, the base V-JEPA predictive objective is

$$\mathcal{L}_{\text{JEPA}} = \mathbb{E}_x \left[\| g_\phi(f_\theta(M_c \odot x), M_t) - \text{sg}(f_{\bar{\theta}}(M_t \odot x)) \|_2^2 \right].$$

Our key decision is that this objective remains ***the dominant optimization target***. Shared adapters modify the ***trainable subspace*** of continual learning, not the semantics of the self-supervised prediction task.

10.4.2 Shared adapters as the continual adaptation subspace

Let the original V-JEPA parameters be partitioned as

$$\Theta = \Theta_{\text{shared}} \cup \Theta_{\text{adapt}},$$

where Θ_{shared} denotes the large backbone and predictor parameters that are frozen or updated only minimally, and Θ_{adapt} denotes a compact collection of trainable adapter-style parameters. The central design choice is that Θ_{adapt} is ***shared across continual sessions***:

$$\Theta_{\text{adapt}}^{(1)} \to \Theta_{\text{adapt}}^{(2)} \to \cdots \to \Theta_{\text{adapt}}^{(T)},$$

rather than creating a completely separate full model for each session.

Why this matters. Vanilla continual fine-tuning distributes new gradients across a very large parameter space, which makes forgetting and uncontrolled drift more likely. Shared adapter tuning instead constrains the update to a low-dimensional, reusable adaptation manifold. SSIAT's empirical message is closely aligned with this view: ***adapter tuning is a strong continual-learning substrate*** and can outperform prompt-based alternatives without requiring parameter expansion at each session. SAPT complements this from the LLM side by showing that reusable PET modules can be learned and selected in a coordinated way, rather than treated as disconnected task-local patches.

10.4.3 Minimal shared-adapter formulation for V-JEPA

In our setting, the simplest instantiation is:

$$f_\theta^{\text{adapt}} = f_{\Theta_{\text{shared}},\,\Theta_{\text{adapt}}},$$

where Θ_{adapt} may correspond to adapter bottlenecks, LoRA-style low-rank updates, SSF-style feature scaling/shifting blocks, or a similarly compact PEFT parameterization.

At session t, we then optimize

$$\min_{\Theta_{\text{adapt}},\,\phi} \quad \mathcal{L}_{\text{JEPA}}(\mathcal{D}_t),$$

while keeping most of Θ_{shared} fixed or updated at a much smaller learning rate. This yields the key property

$$\underbrace{\text{prediction task}}_{\text{unchanged}} \qquad \text{vs.} \qquad \underbrace{\text{adaptation subspace}}_{\text{restricted and shared}}.$$

Interpretation. The model is still solving the same JEPA prediction problem. What changes is that continual learning must now pass through a compact, persistent adapter space. This makes adaptation more controlled, more memory-efficient, and easier to compare across sessions than full-model fine-tuning.

10.4.4 Why shared adapters can be better than session-wise expansion

A natural temptation in continual PEFT is to add a new prompt, adapter, or expert at every session. While that can improve flexibility, it also increases memory, complicates inference, and can fragment knowledge across too many task-local modules. SSIAT is notable precisely because it shows that ***one can incrementally tune a shared adapter*** and still obtain strong continual-learning performance, without imposing parameter-update constraints or expanding a new adapter at every stage. SAPT adds a complementary lesson: when multiple PET blocks or adaptation pathways exist, ***their learning and their selection should be aligned*** rather than optimized independently.

The philosophical shift. This leads to the right interpretation for our setting:

> ***continual adaptation should accumulate in a reusable adapter space, not scatter across the full backbone or proliferate into an ever-growing list of disconnected task-specific modules.***

10.4.5 A shared-adapter view of stability and plasticity

Shared adapter tuning offers a particularly clean compromise between retention and plasticity. The largely frozen backbone preserves broad visual structure and general predictive priors, while the adapter space provides the flexibility to absorb domain-specific shift. In that sense, the adapter acts as a ***controlled interface*** between the stable foundation representation and the continually changing data stream.

A useful way to write this is

$$h(x) = f_{\Theta_{\text{shared}}}(x) + \Delta_{\Theta_{\text{adapt}}}(x),$$

where $f_{\Theta_{\text{shared}}}(x)$ captures the stable pre-trained representation and $\Delta_{\Theta_{\text{adapt}}}(x)$ captures the learned continual correction. The advantage is immediate: the correction term is compact, reusable, and easier to regularize than a full backbone rewrite.

Why this fits V-JEPA especially well. Because V-JEPA already learns in representation space rather than through explicit reconstruction, it is particularly valuable to avoid unnecessary drift in the encoder. Shared adapter tuning preserves this geometry better than unconstrained full-model continual fine-tuning, because adaptation is forced to occur through a compact subspace layered on top of the original predictive machinery.

10.4.6 What we borrow from SSIAT

From SSIAT, the main lesson we adopt is that ***adapter tuning should be treated as a first-class continual learning mechanism in its own right***, not merely as a lightweight engineering shortcut. The paper revisits PEFT methods in continual learning and finds that adapter tuning is superior to prompt-based methods in its setting, even without per-session parameter expansion. It further motivates ***incrementally tuning the shared adapter*** so that semantic shift is absorbed by a common adaptation space rather than by a growing list of isolated prompts.

In our V-JEPA setting, this translates to:

- attach shared adapters to the predictor and upper encoder blocks,
- keep those adapters persistent across sessions,
- continue updating the same adapter set as the continual stream evolves,
- avoid immediately resorting to session-wise adapter expansion.

10.4.7 What we borrow from SAPT

From SAPT, the main lesson we adopt is that ***learning the adapter space and deciding how it is used should not be treated as separate problems***. SAPT introduces a ***Shared Attentive Learning & Selection*** mechanism to align PET learning and PET selection, and demonstrates strong performance on continual learning benchmarks for large language models. Even though our setting is not language modeling, the underlying idea is highly relevant: if multiple adapter channels, blocks, or low-rank components are available, then their ***importance and usage*** should be coordinated rather than left implicit.

In our case, this suggests a natural refinement beyond the simplest shared-adapter baseline:

$$\Delta_{\Theta_{\text{adapt}}}(x) = \sum_{k=1}^{K} \alpha_k(x)\, \Delta_k(x),$$

where $\Delta_k(x)$ are adapter sub-components and $\alpha_k(x)$ are input-conditioned routing or weighting coefficients. Even if we begin with a single shared adapter, SAPT motivates a later extension in which ***adapter usage is made explicit and data-dependent*** rather than uniform across all inputs.

10.4.8 How we will use it in practice

Our concrete shared-adapter plan is the following:

1. start from a strong pre-trained V-JEPA checkpoint;
2. freeze most of the backbone and predictor, or update them only with a very small learning rate;
3. insert shared adapters or LoRA-style modules in the predictor and upper transformer blocks;
4. train these shared modules across all continual sessions using the original JEPA objective;
5. avoid per-session adapter growth in the first round of experiments;

6. optionally add lightweight routing or importance weighting across adapter components if uniform shared adaptation proves insufficient.

This gives a natural experimental ladder:

full continual FT $\longrightarrow$ shared adapter continual tuning $\longrightarrow$ shared adapter tuning with selective routing / weighting.

10.4.9 Why shared adapter continual tuning is a good fit for this paper

Shared adapter continual tuning is especially attractive for the present stage for four reasons.

1. ***It preserves the core JEPA objective.*** We do not need to redefine the prediction task; we only change where continual adaptation lives.
2. ***It is parameter-efficient and implementation-friendly.*** Adapter-style PEFT is substantially lighter than full-model continual fine-tuning and fits naturally into existing foundation-model pipelines.
3. ***It reduces uncontrolled drift.*** By restricting updates to a compact shared subspace, it offers a more stable adaptation route than rewriting the full encoder.
4. ***It gives a clean middle ground between rigidity and proliferation.*** It is more flexible than freezing the entire model, but far cleaner than adding a completely separate adaptation module for every session.

In short. For us, shared adapter continual tuning is best understood not as "attach a small module because it is cheap," but as

> ***keep the JEPA objective, keep the backbone largely stable, and let continual knowledge accumulate inside a reusable adapter space.***

Bottom line. This makes SSIAT / SAPT-style shared adapter continual tuning an especially compelling third alternative to vanilla continual fine-tuning: it is ***parameter-efficient***, ***stable***, and conceptually aligned with our goal of improving continual adaptation ***without abandoning the core V-JEPA predictive objective***. SSIAT contributes the strong empirical case for ***incrementally tuning a shared adapter*** in continual vision settings, while SAPT contributes the deeper principle that ***adapter learning and adapter usage should be aligned rather than decoupled***.

References

Anthropic. 2023. Towards monosemanticity: Decomposing language models with dictionary learning. https://www.anthropic.com/research/towards-monosemanticity-decomposing-language-models-with-dictionary-learning.

Nora Belrose, Zach Furman, Logan Smith, Guy Halawi, Amir Ostrovsky, Lev McKinney, Stella Biderman, and Jacob Steinhardt. 2023. Leace: Perfect linear concept erasure in closed form. *arXiv preprint arXiv:2306.03819*.

Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nicholas L. Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E. Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. 2023. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*. https://transformer-circuits.pub/2023/monosemantic-features/index.html.

Center for AI Safety and collaborators. 2025. Humanity's last exam. https://huggingface.co/datasets/cais/hle. Dataset card and benchmark website.

Yanai Elazar, Shauli Ravfogel, Alon Jacovi, and Yoav Goldberg. 2021. Amnesic probing: Behavioral explanation with amnesic counterfactuals. *Transactions of the Association for Computational Linguistics*, 9:160–175.

Denis Emelin, Ronan Le Bras, Jack Hwang, Maxwell Forbes, and Yejin Choi. 2021. Moral stories: Situated reasoning about norms, intentions, actions, and their consequences. In *Proceedings of EMNLP*.

Maxwell Forbes, Maarten Sap, Ethan Escott, Noah A. Smith, and Yejin Choi. 2021. Social chemistry 101: Learning to reason about social and moral norms. *Transactions of the Association for Computational Linguistics*, 9:26–44.

Atticus Geiger, Sarah Rubin, Spencer Qiu, Keyon Vafa, Chris Dyer, Christopher Potts, and Dan Jurafsky. 2023. Towards understanding mechanistic interpretability via causal abstraction. *arXiv preprint arXiv:2301.04709*.

Dan Hendrycks, Collin Burns, Steven Basart, Andrew Critch, Jerry Li, Dawn Song, and Jacob Steinhardt. 2021. Aligning ai with shared human values. *Proceedings of ICLR*.

John Hewitt and Percy Liang. 2019. Designing and interpreting probes with control tasks. In *Proceedings of EMNLP-IJCNLP*.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, et al. 2019. Parameter-efficient transfer learning for nlp. In *ICML*.

Robert Huben, Hoagy Cunningham, Logan Riggs Smith, Aidan Ewart, and Lee Sharkey. 2024. Sparse autoencoders find highly interpretable features in language models. In *International Conference on Learning Representations*.

Menglin Jia, Luming Tang, Bor-Chun Chen, et al. 2022. Visual prompt tuning. In *ECCV*.

Abhinav Kumar and Chenhao Tan. 2022. Probing classifiers are unreliable for concept removal and detection. In *Advances in Neural Information Processing Systems*.

Dongze Lian, Daquan Zhou, Jiashi Feng, and Xinchao Wang. 2022. Scaling & shifting your features: A new baseline for efficient model tuning. In *NeurIPS*.

Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. 2022. Locating and editing factual associations in gpt. In *Advances in Neural Information Processing Systems*.

OpenAI. 2024. Scaling and evaluating sparse autoencoders. https://cdn.openai.com/papers/sparse-autoencoders.pdf.

Shauli Ravfogel, Yanai Elazar, Hila Gonen, Michael Twiton, and Yoav Goldberg. 2020. Null it out: Guarding protected attributes by iterative nullspace projection. In *Proceedings of ACL*.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, Samuel R. Bowman, et al. 2023. Gpqa: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*.

Nina Rimsky, Yair Gabrieli, Oded Obeso, Neel Nanda, and Marius Arora. 2024. Steering llama 2 via contrastive activation addition. *arXiv preprint arXiv:2312.06681*.

Nino Scherrer, Felix Shi, Amir Feder, and David Blei. 2023. Moralchoice: A benchmark for moral decision-making and alignment with human values. *arXiv preprint arXiv:2307.14324*.

Elena Voita and Ivan Titov. 2020. Information-theoretic probing with minimum description length. In *Proceedings of EMNLP*.

Colin White, Mert Safari, Yash Singhal, et al. 2024. Livebench: A challenging, contamination-free llm benchmark. *arXiv preprint arXiv:2406.19314*.

Fred Zhang and Neel Nanda. 2023. Towards best practices of activation patching in language models: Metrics and methods. *arXiv preprint arXiv:2309.16042*.

Linglan Zhao, Xuerui Zhang, Ke Yan, Shouhong Ding, and Weiran Huang. 2024. Safe: Slow and fast parameter-efficient tuning for continual learning with pre-trained models. In *Advances in Neural Information Processing Systems*.

Noah Ziems, Shaily Bhatt, Yejin Choi, and Diyi Yang. 2023. Normbank: A knowledge bank of situational social norms. *arXiv preprint arXiv:2305.17008*.